

رسالة محمد

یادگیری ماشین

مدرس: محمدرضا محمدی

بهار ۱۴۰۲

شبکه‌های عصبی کانولوشنی

Convolutional Neural Networks

- Metrics
- Losses
- Built-in small datasets
- Keras Applications
- Utilities
- Code examples
- Why choose Keras?
- Community & governance
- Contributing to Keras

Available models

Model	Size	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth
Xception	88 MB	0.790	0.945	22,910,480	126
VGG16	528 MB	0.713	0.901	138,357,544	23
VGG19	549 MB	0.713	0.900	143,667,240	26
ResNet50	98 MB	0.749	0.921	25,636,712	-
ResNet101	171 MB	0.764	0.928	44,707,176	-
ResNet152	232 MB	0.766	0.931	60,419,944	-
ResNet50V2	98 MB	0.760	0.930	25,613,800	-
ResNet101V2	171 MB	0.772	0.938	44,675,560	-
ResNet152V2	232 MB	0.780	0.942	60,380,648	-
InceptionV3	92 MB	0.779	0.937	23,851,784	159
InceptionResNetV2	215 MB	0.803	0.953	55,873,736	572
MobileNet	16 MB	0.704	0.895	4,253,864	88
MobileNetV2	14 MB	0.713	0.901	3,538,984	88
DenseNet121	33 MB	0.750	0.923	8,062,504	121
DenseNet169	57 MB	0.762	0.932	14,307,880	169
DenseNet201	80 MB	0.773	0.936	20,242,984	201
NASNetMobile	23 MB	0.744	0.919	5,326,716	-

Cars Dataset



Overview

The *Cars* dataset contains 16,185 images of 196 classes of cars. The data is split into 8,144 training images and 8,041 testing images, where each class has been split roughly in a 50-50 split. Classes are typically at the level of *Make*, *Model*, *Year*, e.g. 2012 Tesla Model S or 2012 BMW M3 coupe.

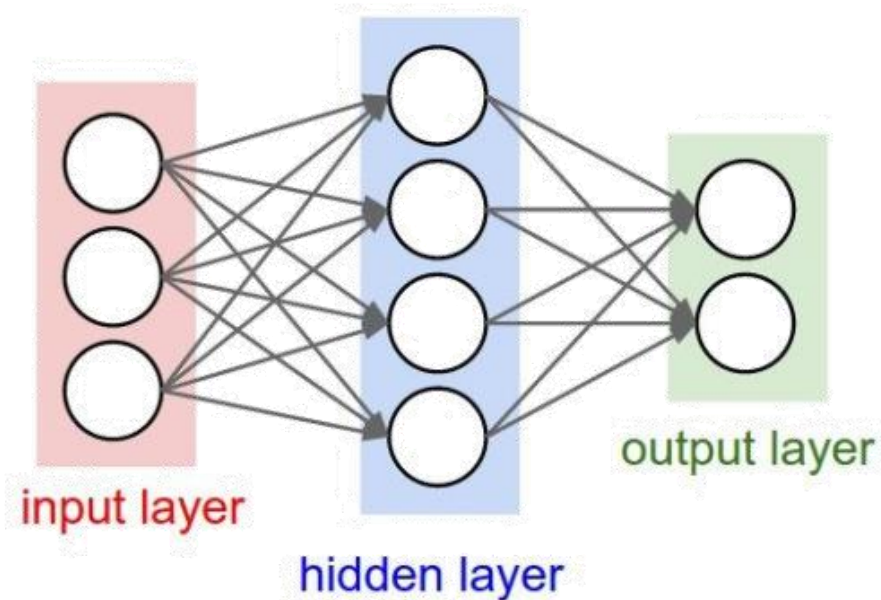
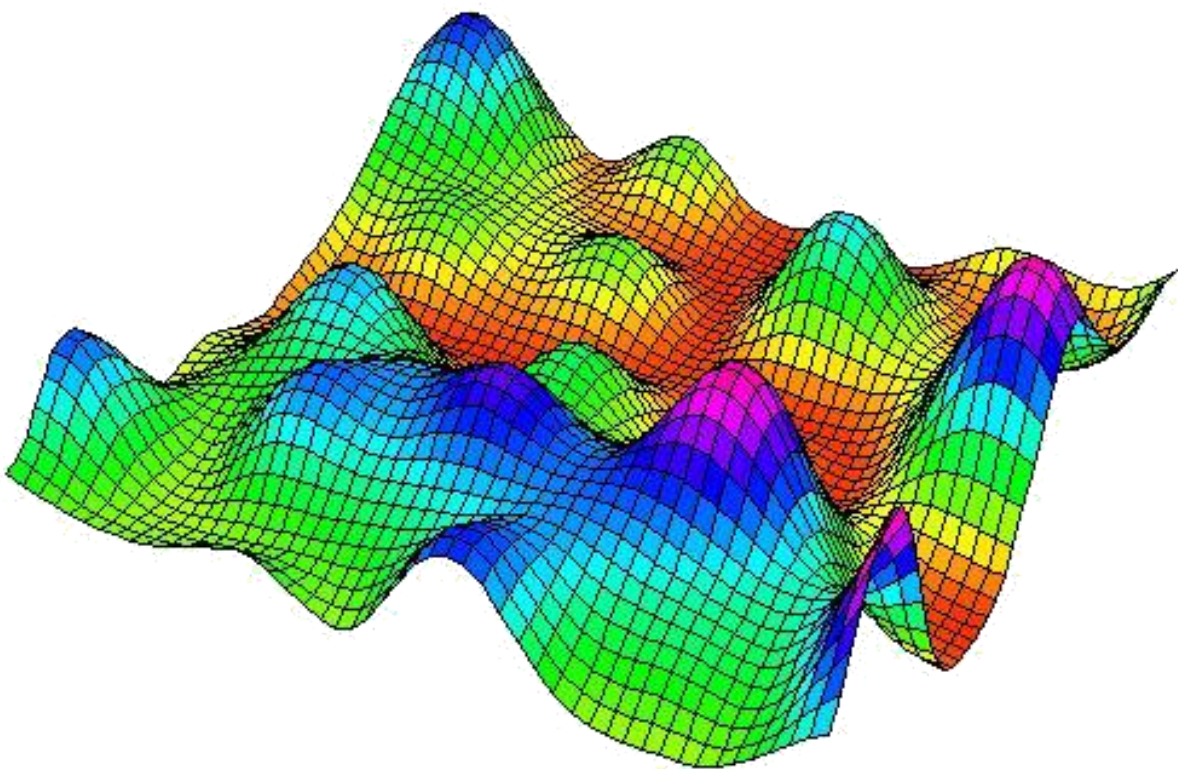


مقداردهی اولیه وزن‌ها

Weight Initialization

مقداردهی اولیه

- در روش‌های بهینه‌سازی مبتنی بر تکرار، نقطه شروع بهینه‌سازی بسیار مهم است
- اگر در ابتدای کار تمام وزن‌های شبکه مقدار صفر داشته باشند چه اتفاقی می‌افتد؟



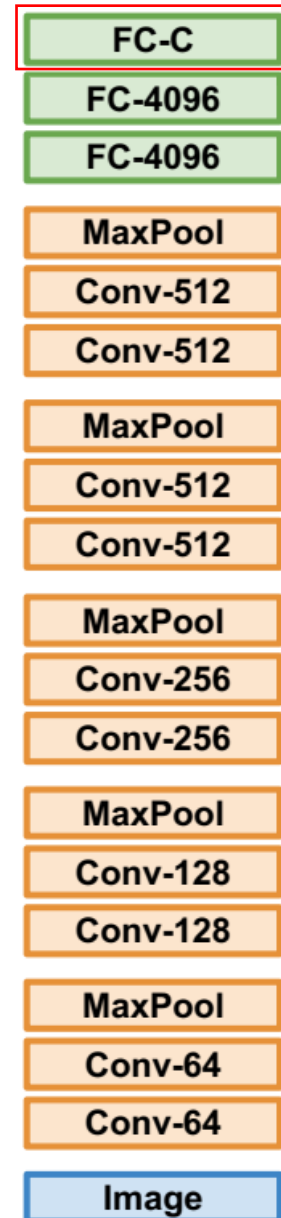
مقداردهی اولیه

- روش مقداردهی اولیه Xavier یکی از معروفترین روش‌های وزن‌دهی اولیه است
- مقداردهی اولیه مناسب هنوز یک زمینه تحقیقاتی فعال است
- به خصوص برای مجموعه داده‌های کوچک، مقداردهی اولیه بسیار حائز اهمیت است
- برای آموزش یک شبکه CNN با میلیون‌ها پارامتر، حجم زیادی از داده‌های آموزشی لازم است
- با استفاده از تقویت داده و دیگر روش‌های تنظیم پارامترهای شبکه می‌توان تا حدی کمبود داده را جبران کرد
- یکی از بهترین روش‌ها برای مقداردهی اولیه پارامترهای یک شبکه استفاده از شبکه‌های pretrained است
- انتقال یادگیری روش بسیار موثری است تا دانش بدست آمده توسط یک شبکه به شبکه جدید منتقل شود

Train on ImageNet



Small Dataset (C classes)



Reinitialize
this and train

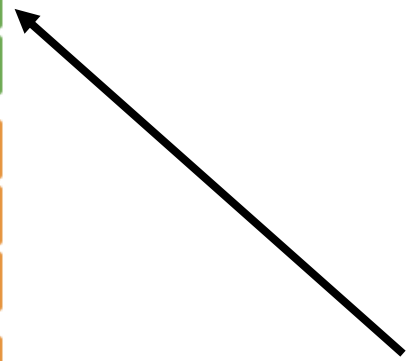
Freeze
these

Bigger Dataset (C classes)

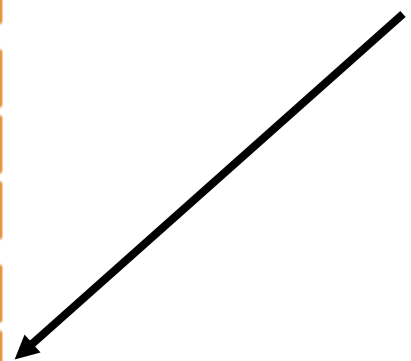


Train these

Freeze
these



More specific



More generic



مجموعه داده بسیار متفاوت	مجموعه داده بسیار مشابه	
?	?	مجموعه داده خیلی کم
?	?	مجموعه داده زیاد



مجموعه داده بسیار متفاوت	مجموعه داده بسیار مشابه	
?	از یک دسته‌بند خطی در آخرین لایه استفاده شود	مجموعه داده خیلی کم
?	?	مجموعه داده زیاد



مجموعه داده بسیار متفاوت	مجموعه داده بسیار مشابه	
?	از یک دسته‌بند خطی در آخرین لایه استفاده شود	مجموعه داده خیلی کم
?	تعدادی از لایه‌های انتهای تنظیم دقیق شوند	مجموعه داده زیاد



مجموعه داده بسیار متفاوت	مجموعه داده بسیار مشابه	
مشکل است! می توان دسته بند خطی را در گام های مختلف امتحان کرد	از یک دسته بند خطی در آخرین لایه استفاده شود	مجموعه داده خیلی کم
?	تعدادی از لایه های انتهایی تنظیم دقیق شوند	مجموعه داده زیاد



مجموعه داده بسیار متفاوت	مجموعه داده بسیار مشابه	
مشکل است! می توان دسته بند خطی را در گام های مختلف امتحان کرد	از یک دسته بند خطی در آخرین لایه استفاده شود	مجموعه داده خیلی کم
تعداد زیادی از لایه های انتهایی تنظیم دقیق شوند	تعدادی از لایه های انتهایی تنظیم دقیق شوند	مجموعه داده زیاد

انتقال یادگیری

- در صورتیکه مجموعه داده‌های شما به اندازه کافی بزرگ نیست و مسئله پیچیده است (شبکه دارای پارامترهای زیادی است):
 - یک مجموعه داده بسیار بزرگ که به مجموعه داده مورد نظر مشابه است انتخاب و شبکه کانولوشنی با آن آموزش ببیند
 - انتقال یادگیری به مجموعه داده مورد نظر انجام شود
- خوشبختانه مدل‌های پیش‌آمोخته زیادی در دسترس هستند

PyTorch: <https://github.com/pytorch/vision>

TensorFlow: <https://github.com/tensorflow/models>

Caffe: <https://github.com/BVLC/caffe/wiki/Model-Zoo>

MatConvNet: <http://www.vlfeat.org/matconvnet/pretrained/>

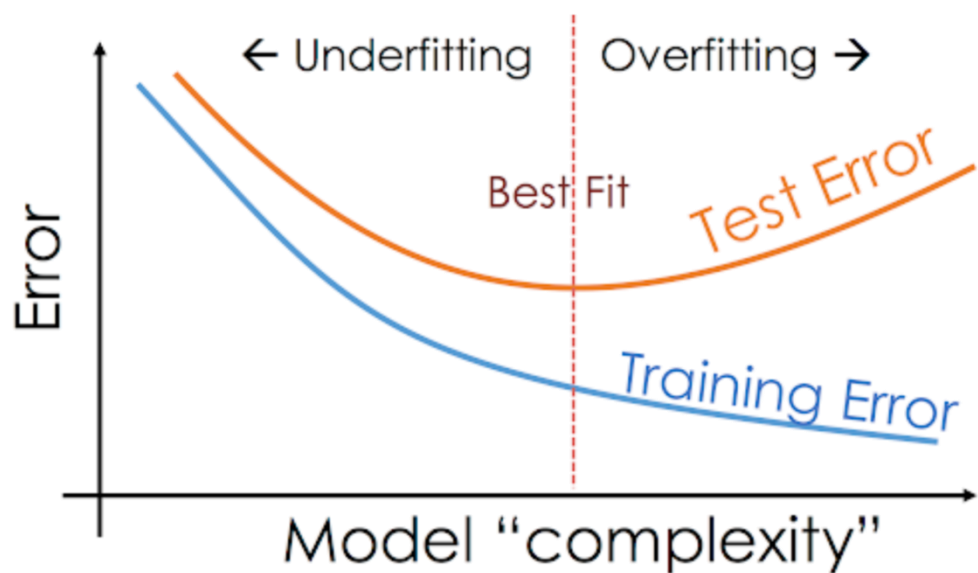
Keras: <https://github.com/fchollet/deep-learning-models/releases/>

منظم سازی

Regularization

منظم‌سازی

- یک مسئله اساسی در یادگیری ماشین این است که چگونه الگوریتمی بسازیم که نه تنها بر روی داده‌های آموزشی بلکه برای ورودی‌های جدید نیز به خوبی عمل کند
- بسیاری از استراتژی‌های یادگیری ماشین به دنبال کاهش خطای آزمون هستند حتی اگر به افزایش خطای آموزش منجر شود



- به این استراتژی‌ها regularization (منظم‌سازی) گفته می‌شود

بیش بر ارزش

- در سال ۲۰۱۷ گروهی از پژوهشگران نشان دادند که شبکه‌های عمیق با ظرفیت یادگیری بسیار بالا، می‌توانند حتی تصاویر با برجسب‌های تصادفی را آموزش ببینند
- علی‌رغم عدم وجود الگوی واقعی که ورودی‌ها را به خروجی‌ها مرتبط کند، پارامترهای شبکه عصبی می‌تواند بهینه شود

3	8	6	9	6	4	5	3	8	4	5	2	3	8	4	8
1	5	0	5	9	7	4	1	0	3	0	6	2	9	9	4
1	3	6	8	0	7	7	6	8	9	0	3	8	3	7	7
8	4	4	1	2	9	8	1	1	0	6	6	5	0	1	1
7	2	7	3	1	4	0	5	0	6	8	7	6	8	9	9
4	0	6	1	9	2	8	3	7	4	1	5	6	6	1	7
2	8	6	9	7	0	9	1	6	2	8	3	6	4	9	5
8	6	8	7	8	8	6	9	1	7	6	0	9	6	7	0

- برای حالت ۱۰ کلاسه تا ۹۰٪ فاصله بین آموزش و آزمون می‌تواند وجود داشته باشد

جریمه اندازه پارامترها

- به منظور محدود کردن پیچیدگی مدل، می‌توانیم محدودیت‌هایی بر روی پارامترهای مدل اعمال کنیم
- تابعی که خروجی آن یک عدد ثابت برای همه ورودی‌ها باشد، ساده‌ترین تابع است و می‌توانیم میزان پیچیدگی یک تابع را در مقایسه با آن بسنجیم

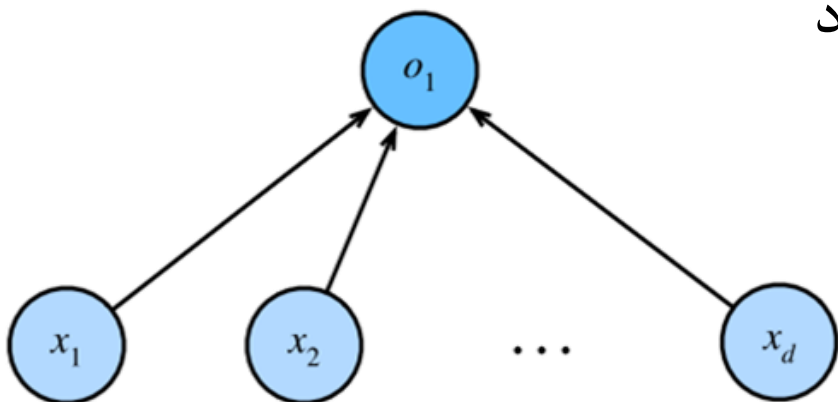
- فرض کنید $L(\mathbf{w}, b)$ یکی از توابع ضرر باشد که خطای پیش‌بینی $\hat{y}^{(i)}$ ‌ها را محاسبه می‌کند

- تابع ضرر جدید با اعمال جریمه بر روی پارامترها: $\tilde{L}(\mathbf{w}, b) = L(\mathbf{w}, b) + \lambda \Omega(\mathbf{w})$

- معمولاً محدودیتی روی بایاس‌ها (به خصوص لایه آخر) اعمال نمی‌شود

- به λ ثابت منظم‌سازی گفته می‌شود

- با چه رابطهای فاصله تا صفر را اندازه بگیریم؟



منظم‌سازی پارامتر L2

- اگر بجای L2 از فاصله اقلیدوسی استفاده کنیم محاسبه مشتق پیچیده می‌شود

$$\Omega(\mathbf{w}) = \frac{1}{2} \|\mathbf{w}\|_2^2$$

$$\tilde{L}(\mathbf{w}, b) = L(\mathbf{w}, b) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$$

- این استراتژی وزن‌های شبکه را به سمت مبدا نزدیک می‌کند

$$\nabla_{\mathbf{w}} \tilde{L}(\mathbf{w}, b) = \nabla_{\mathbf{w}} L(\mathbf{w}, b) + \lambda \mathbf{w}$$

- گرادیان تابع ضرر منظم‌شده:

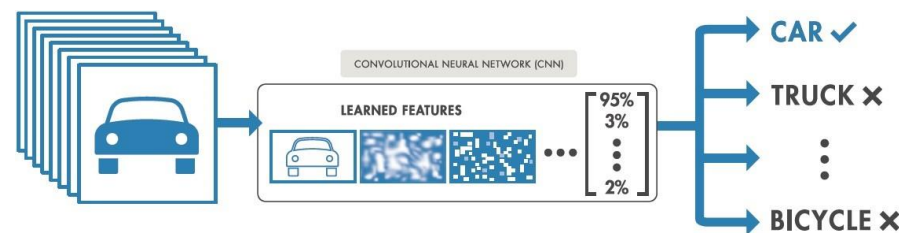
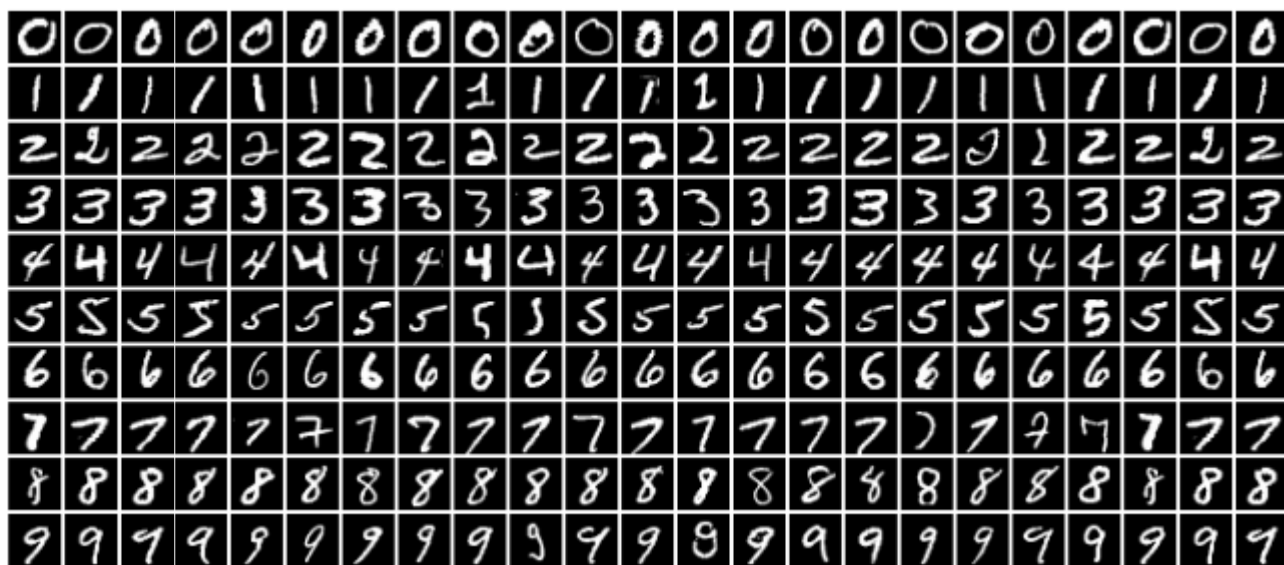
$$\mathbf{w} \leftarrow \mathbf{w} - \eta (\lambda \mathbf{w} + \nabla_{\mathbf{w}} L(\mathbf{w}, b))$$

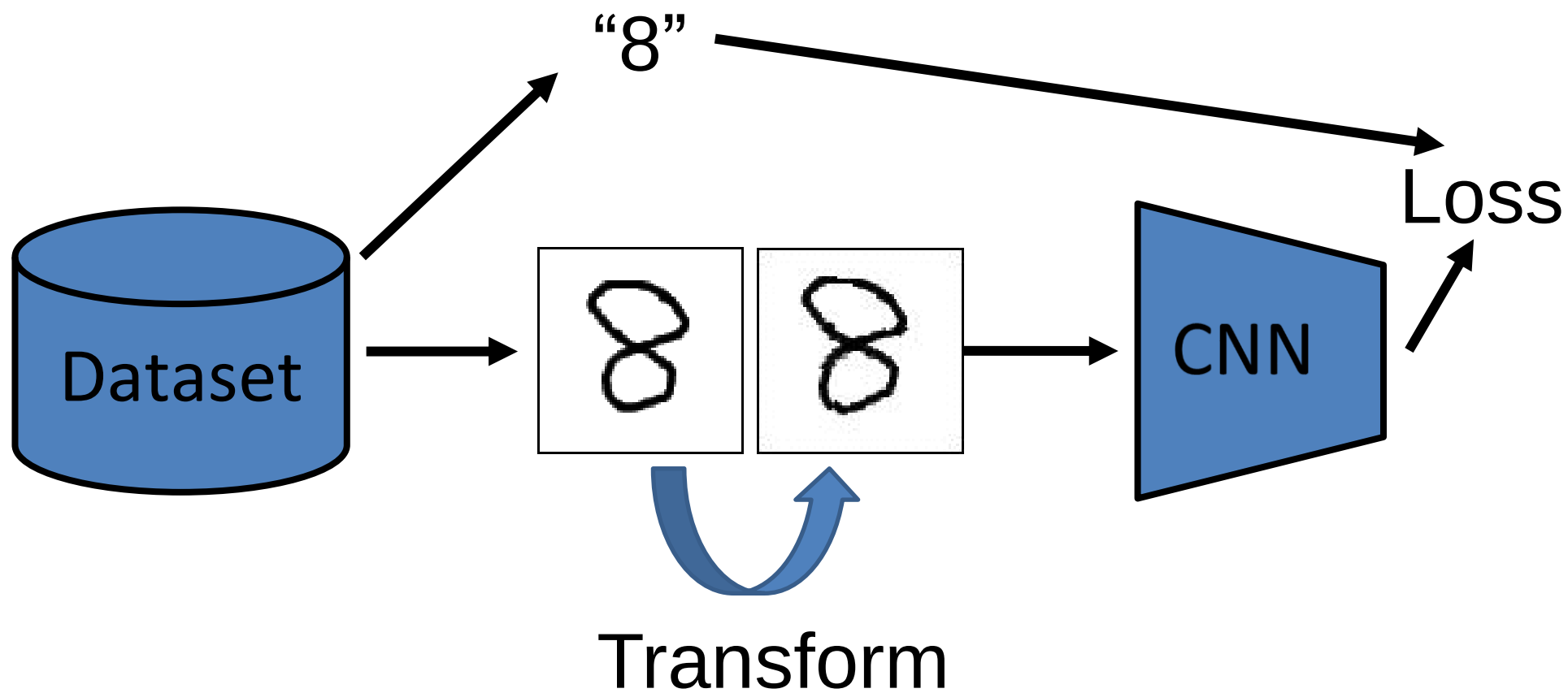
$$\mathbf{w} \leftarrow (1 - \eta \lambda) \mathbf{w} - \eta \nabla_{\mathbf{w}} L(\mathbf{w}, b)$$

- به این روش، Weight Decay هم گفته می‌شود

داده‌افزایی (Data Augmentation)

- بهترین راه برای افزایش قدرت تعمیم‌دهی یک الگوریتم یادگیری ماشین با ظرفیت یادگیری بالا، آموزش آن بر روی داده‌های بیشتر است
- جمع‌آوری داده معمولاً فرآیند دشوار و خسته‌کننده‌ای است
- می‌توانیم داده‌های ساختگی بسازیم و به داده‌های آموزشی اضافه کنیم
- این کار برای مسئله دسته‌بندی راحت‌ترین است
- می‌توانیم جفت‌های (x, y) جدید را به سادگی و تنها با تبدیل x بسازیم





داده‌افزایی: flip



b

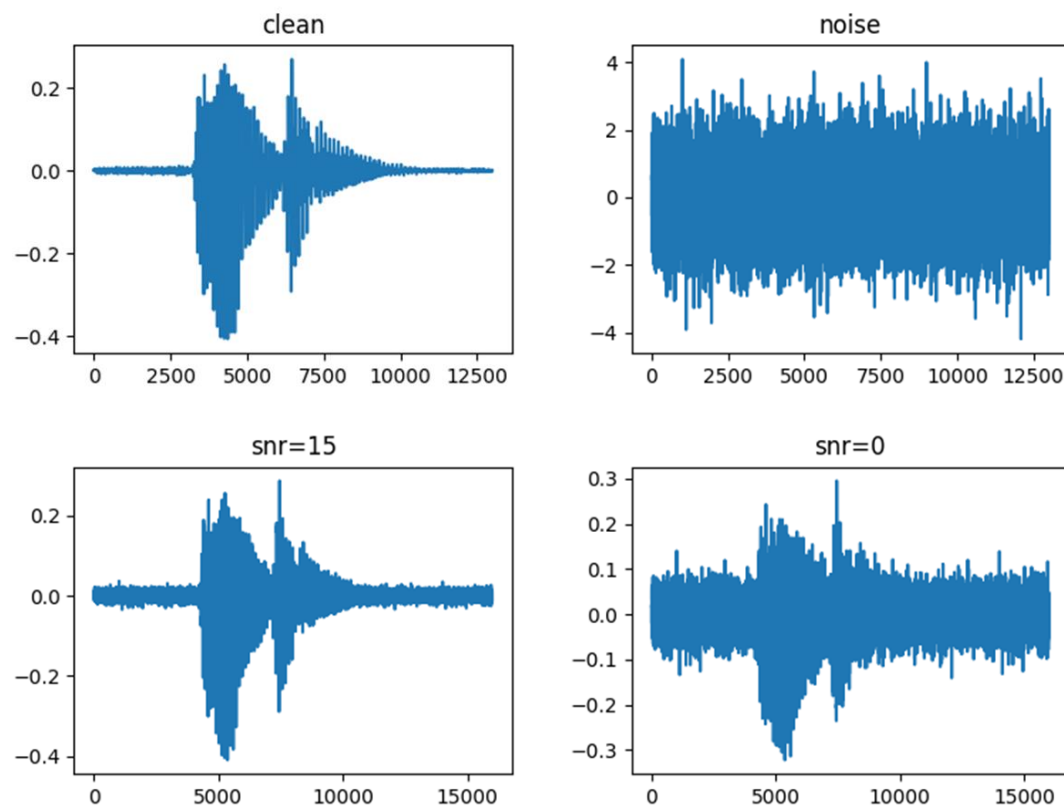
d

p

q

داده‌افزایی: افزودن نویز

- افزودن نویز در ورودی به یک شبکه عصبی می‌تواند به عنوان نوعی داده‌افزایی در نظر گرفته شود



- برای بسیاری از مسائل دسته‌بندی و حتی برخی از مسائل رگرسیون، با افزودن مقدار محدودی نویز به ورودی همچنان می‌توان همان خروجی را توقع داشت
- افزودن نویز می‌تواند در لایه‌های میانی شبکه نیز انجام شود